YOGESH M V

AI ENGINEER | LLM SYSTEMS, RAG & AGENTIC AI
Chennai, India | +91-6383823772 | yogesh.mvdev@gmail.com | linkedin.com/in/yogeshmv | github.com/yogeshmvgd

PROFILE SUMMARY

AI Engineer with 1+ year building production LLM systems for document AI: RAG pipelines with structure-aware retrieval, LangGraph multi-agent workflows and self-hosted LLM deployments on vLLM. Focused on grounded outputs, rigorous evaluation (RAGAS, LLM-as-judge) and cost-efficient inference across OpenAI, Gemini and local models.

PROFESSIONAL EXPERIENCE

Sybrant Technologies | Software Engineer August 2025 – Present

LeaseRush – AI Lease Abstraction Platform

- Owned and extended the extraction pipeline (Docling → per-lease FAISS index → clause retrieval → LLM extraction → Python validation → reviewer workflow): added 15+ clause handlers (rent, dates, options, expenses), a persistent FAISS index and source highlighting.
- Raised key-field capture from ~60% to ~80% (measured by the abstraction team) by replacing fixed-size chunking that split clauses with Docling structure-aware chunking and multi-query retrieval; LLM-as-judge evaluation against reviewer answers on 15–20 leases scored ~75% with local Qwen.
- Cut OpenAI cost per lease from ~Rs. 120–150 to ~Rs. 20–40 (verified against token logs and billing) by switching to structure-aware chunks, retrieving 20 instead of 80 chunks (~2,000 characters each), and migrating from GPT-4o to GPT-5 mini.
- Moved rent-schedule, escalation, free-rent and billing-period calculations out of the LLM into deterministic Python calculators, with grounding checks that drop any rent value not present in the source lease.
- Integrated a self-hosted Qwen 2.5 32B (vLLM/Ollama on an internal GPU server) via a multi-provider LLM layer; leases run fully in-house (~15 min) or on OpenAI.
- Adapted the Celery pipeline for the local LLM: priority queues and a bulk-orchestration queue to prevent worker deadlock, live status via WebSockets.
- Created a RAGAS and LLM-as-judge evaluation harness (per-handler thresholds, token/cost tracking); fixed LLM timeouts silently saved as clause text.
- Deployed and operated the platform on Linux (WSL) with Nginx, Gunicorn, systemd and GitHub Actions CI/CD; Docker for the dev server.

LeaseRush QC – AI Lease Review Agent (partly in production)

- Developing a LangGraph agentic workflow that automates second-level QC of lease abstracts: specialist agents check the abstract against the lease and amendments (OCR for scanned pages) and output severity-ranked findings with page-level evidence.
- Shipped the first checks to production (spacing, abbreviation consistency, spelling and grammar), freeing reviewers to focus on substantive errors; date, dollar-amount and clause checks are in testing.
- Implemented clause-completeness checks: each clause maps to a question set; the agent verifies “silent” clauses against the lease and turns unanswered questions into targeted searches to catch details the abstractor missed.
- Designed an “LLM extracts, Python judges” architecture (the LLM fills Pydantic schemas with page, section and quote; deterministic rules decide pass/fail), currently being validated against past human-reviewer corrections.
- Controlled cost with capped per-agent BM25 retrieval (one step sends 24% of the lease), caching, a per-run budget and batching; tuning reasoning effort cut extraction-step cost by 41–55% with no loss in extracted facts.

PactPro – AI Legal Drafting & Risk Analysis

- Developed a contract generation pipeline with three drafting modes (template, rewrite from sample, clause library), retrieving reference clauses from pgvector and generating clauses in fixed-size batches (6 per call on cloud models, 1 on local models).
- Built privacy masking for client data before it reaches the LLM: names the firm already holds (clients, addresses, case parties, opposing counsel and firm staff) are replaced with typed placeholders such as [CLIENT_1] and [PARTY_1], and draft fact fields are flagged as private based on their field names (e.g. landlord, tenant, party, address). Real names are restored automatically after generation, so nothing downstream ever sees a placeholder.
- Designed playbook-based risk analysis: the LLM extracts clause positions from a firm’s past agreements, embedding clustering decides standard vs fallback positions, and draft clauses are checked for severity-rated risks.
- Engineered a unified LLM provider layer: one interface over OpenAI, Gemini and local models.

PROJECTS

vault-to-voice – Agentic Blog Writer github.com/yogeshmvgd/vault-to-voice

- Developed a LangGraph agent that turns Obsidian notes into blog posts in the author’s voice: interview subgraph and approval steps with human-in-the-loop interrupt(), parallel section drafting (Send), resumable SQLite checkpoints and long-term memory of the author’s style.
- Implemented sentence-level citations to source notes, verified by embedding similarity plus an LLM judge; added a style-match critic, prompt-injection and PII guardrails; served via FastAPI + SSE and Streamlit on Ollama, with CI on GitHub Actions.

SKILLS

LLMs & GenAI: RAG, LangGraph agents, human-in-the-loop, guardrails, Pydantic structured outputs, vLLM, Ollama, OpenAI and Gemini APIs, LangChain
Retrieval & Document AI: FAISS, pgvector, hybrid search (BM25 + dense), Sentence-Transformers, Hugging Face, Docling, PyMuPDF, Tesseract OCR
Evaluation & Observability: RAGAS, LLM-as-judge, pytest
Backend: Django, Django REST Framework, FastAPI, Flask, Celery, Redis, WebSockets (Django Channels), SSE, Streamlit, PostgreSQL (pgvector)
Languages & Tools: Python, SQL, Linux, Docker, Nginx, Gunicorn, systemd, GitHub Actions, Git

EDUCATION

B.Tech – Artificial Intelligence and Machine Learning, Rajalakshmi Engineering College 2021 – 2025